Kinect Lite: Final Report on the Initial Pipeline

Oct 16, 2026 · @Sumanth G

Summary

The initial pipeline turns a noisy depth image into a restored one in two classical steps: a denoising filter, then hole filling. **No model is trained.** Every step is a hand-designed image filter with a few settings, such as a 9 x 9 window or a blur width.

Designed filters, not a learned model. A trained model (a neural network, for example) learns millions of weights from example pairs of noisy and clean images. Here the filters are fixed formulas from image processing: a median, a Gaussian, a bilateral, and inpainting to fill holes. The only data-driven part is **tuning**: trying each filter's few settings on a set of tuning frames and keeping the one with the lowest error. That is a search over a handful of numbers, not training, and it was checked on separate test frames.

This has three practical consequences:

- The pipeline runs on any depth image straight away, with no training data, GPU or model file.
- Its behaviour is explainable: a median removes spikes, a Gaussian averages, inpainting extends nearby depth into holes.
- It cannot learn scene knowledge (for example that walls are flat), which a trained model could; that is a possible later step.

The pipeline's default is the invalid-aware median (9 x 9 window) with depth below 0.4 m removed, then FMM inpainting. This report runs it once over every frame of both datasets and shows what it does.

What we use

Two public datasets, standard image-processing methods, and six Python libraries; no GPU and no model files.

Data

dataset	what it is	role
ICL-NUIM living room kt0	1,509 rendered depth frames with simulated sensor noise, plus the true depth of every pixel	scoring: every result can be checked against the truth
TUM RGB-D fr3 long_office_household	2,509 real depth frames from an Asus Xtion (Kinect-type sensor), plus the true camera path	checking behaviour on a real sensor

Methods

step	methods
gate	depth below 0.4 m treated as missing
denoising	median, invalid-aware median, Gaussian, bilateral (OpenCV and masked versions)
hole filling	FMM inpainting (Telea), Navier-Stokes inpainting, region fill (median of neighbours)
scoring	mean absolute error against true depth, on flat areas and edge bands separately
3D view	pinhole camera back-projection to point clouds

Software (Python 3.12, all from `pip`)

package	used for
NumPy 2.5	arrays and the masked filters
OpenCV 5.0	image I/O, Gaussian and bilateral filters, inpainting, video
SciPy 1.18	median filter, hole labelling
pandas 3.0	result tables
Matplotlib 3.11	figures

package	used for
Plotly 7.1	interactive 3D viewers

How the pipeline works

Each frame passes through four steps on its own; frames do not share information.

1. **Read the depth image.** A 16-bit PNG (value / 5000 = metres, 0 = missing), a frame of an 8-bit depth video, or a dataset frame. Missing pixels become NaN.
2. **Gate.** Depth below 0.4 m is set to missing: a Kinect-type sensor cannot measure that close, so such values are errors. On ICL they are 0.5 % of pixels but 25 % of the error.
3. **Denoise.** Default: the invalid-aware median with a 9 x 9 window. Each measured pixel becomes the median of the measured pixels around it, which removes spikes and keeps edges sharp. Holes are left as holes. Median, Gaussian and bilateral are options.
4. **Fill holes.** Default: FMM inpainting (Telea), which grows each hole's surrounding depth inward from the edge. Every filled value is clipped to the depth range around its own hole, because FMM can overshoot. Navier-Stokes inpainting and a region fill are options.

The result is a restored depth image with no missing pixels, saved in the input's format. Measured pixels are only changed by step 3; step 4 only writes into holes.

Settings and where they came from

setting	value	source
gate	0.4 m	sensor's minimum range
filter	invalid-aware median, 9 x 9	lowest error of all filters on 31 ICL tuning frames, confirmed on 30 test frames
fill	FMM, radius 5 px	project plan; Navier-Stokes filled more accurately on ICL (33 vs 130 mm)

The full run

The whole pipeline was run once, with its default settings, over every frame of both datasets: 4,018 frames in 7 minutes on this PC.

	ICL-NUIM	TUM fr3
frames	1,509 (all)	2,509 (all)
settings	0.4 m gate, invalid-aware median 9 x 9, FMM fill	same
total time (4 parallel workers)	151 s	270 s
time per frame (one worker)	343 ms median	397 ms median
true depth available	yes: error measured on every frame	no

Commands:

```
python denoise/process.py --dataset icl --frames 0:1509:1 --out
outputs/process/icl_full --no-pngs --fps 30 --video-scale 0.5 --show-frame
775
python denoise/process.py --dataset tum --frames 0:2509:1 --out
outputs/process/tum_full --no-pngs --fps 30 --video-scale 0.5 --show-frame
1200
```

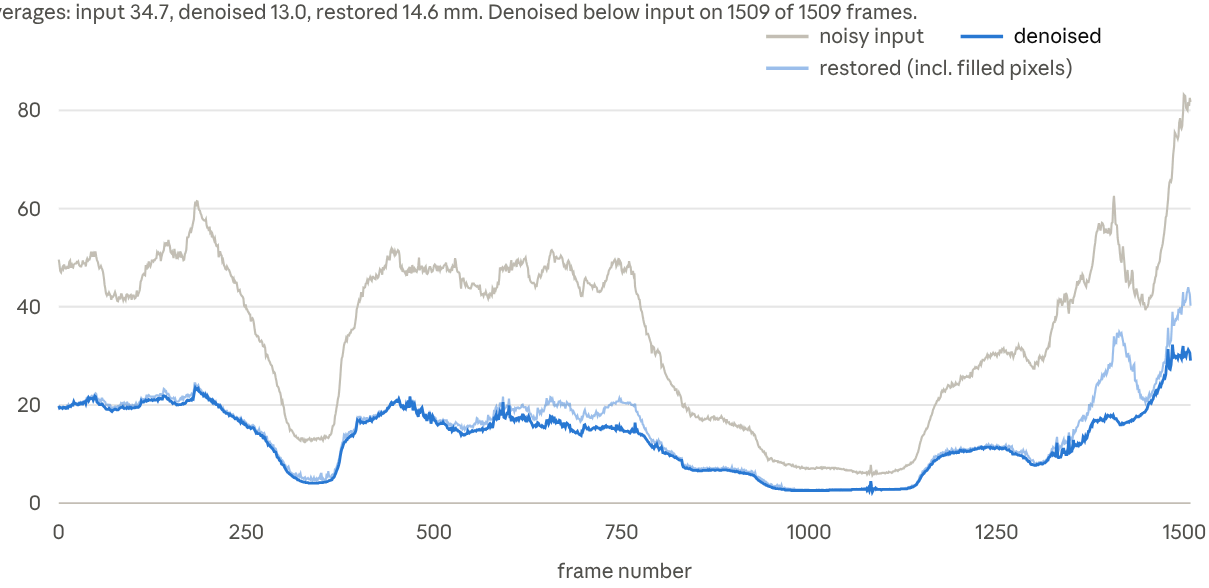
Each run wrote a 30 fps preview video of every frame side by side (`preview.mp4` , 39-49 MB), per-frame numbers (`frames.csv`), a summary plot, a stage figure and a 3D viewer of one frame. The restored PNGs were not kept for this run (`--no-pngs`); without that flag they are saved too.

Results of the full run

On ICL, denoising cut the average error from 34.7 mm to 13.0 mm and lowered it on all 1,509 frames; filling every hole then raised it slightly, to 14.6 mm.

Denoising lowered the error on every one of the 1,509 ICL frames

Mean |error| per frame against the true depth (mm), invalid-aware median 9 x 9 + FMM fill, 0.4 m gate.
Averages: input 34.7, denoised 13.0, restored 14.6 mm. Denoised below input on 1509 of 1509 frames.



outputs/process/icl_full/frames.csv · all 1,509 ICL frames

The error rises and falls with the view: frames with many object edges in sight are hardest for both the input and the filter.

ICL, averages over 1,509 frames

	missing pixels	mean error (mm)
noisy input	0.53 %	34.7
after the gate and denoising	1.02 %	13.0
restored (holes filled)	0 %	14.6
the filled pixels alone (3,128 per frame)	-	132

The gate adds holes (0.53 % to 1.02 %) by removing values below 0.4 m, which are errors. The filled pixels are far less accurate than measured ones, because most holes sit on object edges or where the gate removed a wrong value.

TUM, 2,509 real frames

	value
missing pixels in the input	16.6 % on average (11.7-26.9 % per frame)

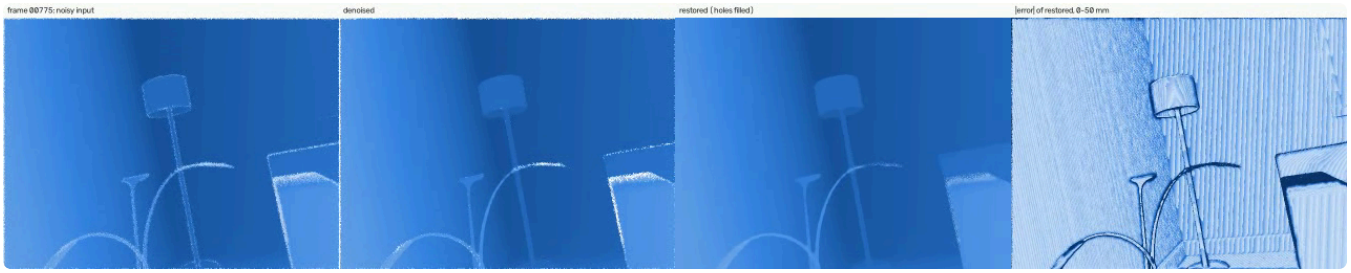
	value
after denoising	unchanged: the filter never fills or deletes
after filling	0 %
pixels filled per frame	51,108 on average (36,000-82,500)

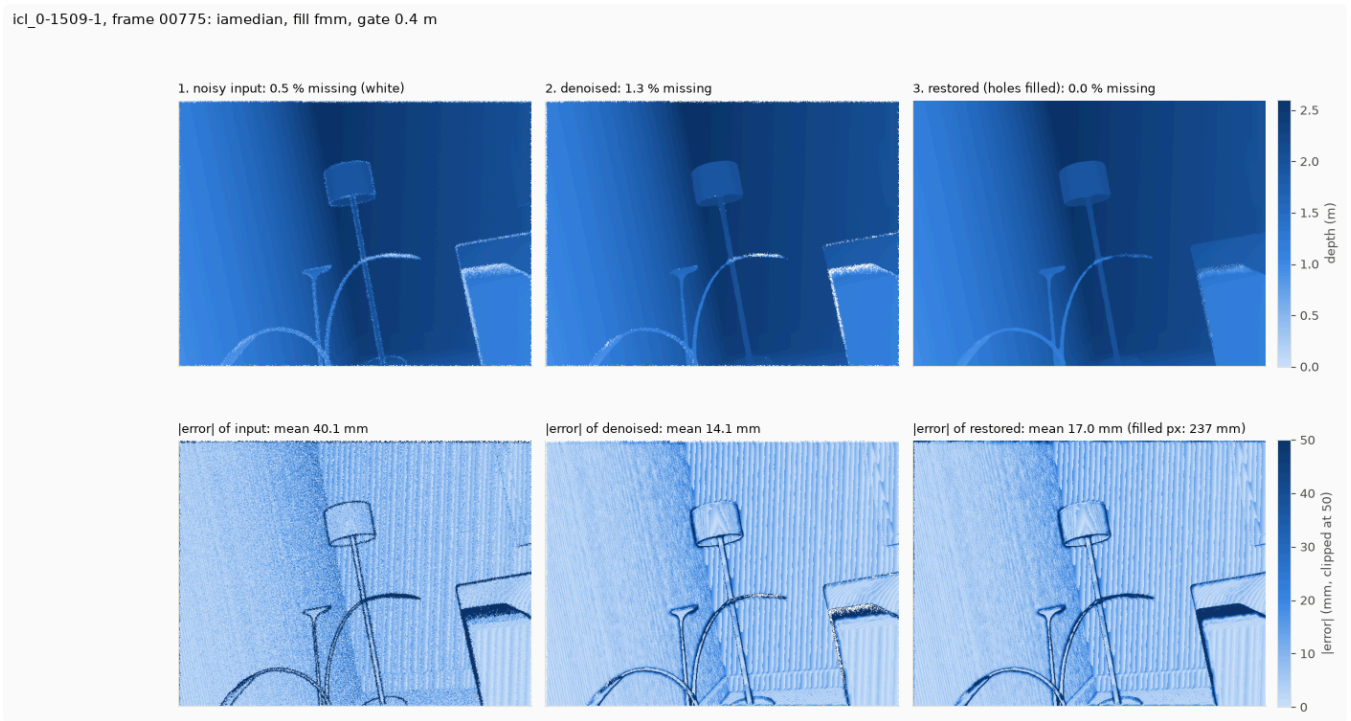
Without true depth on TUM, the fills cannot be scored; the 3D views below show what they look like.

Visual results

The preview videos show the pipeline on every frame; the stills below are one frame from each, then each stage of that frame in detail.

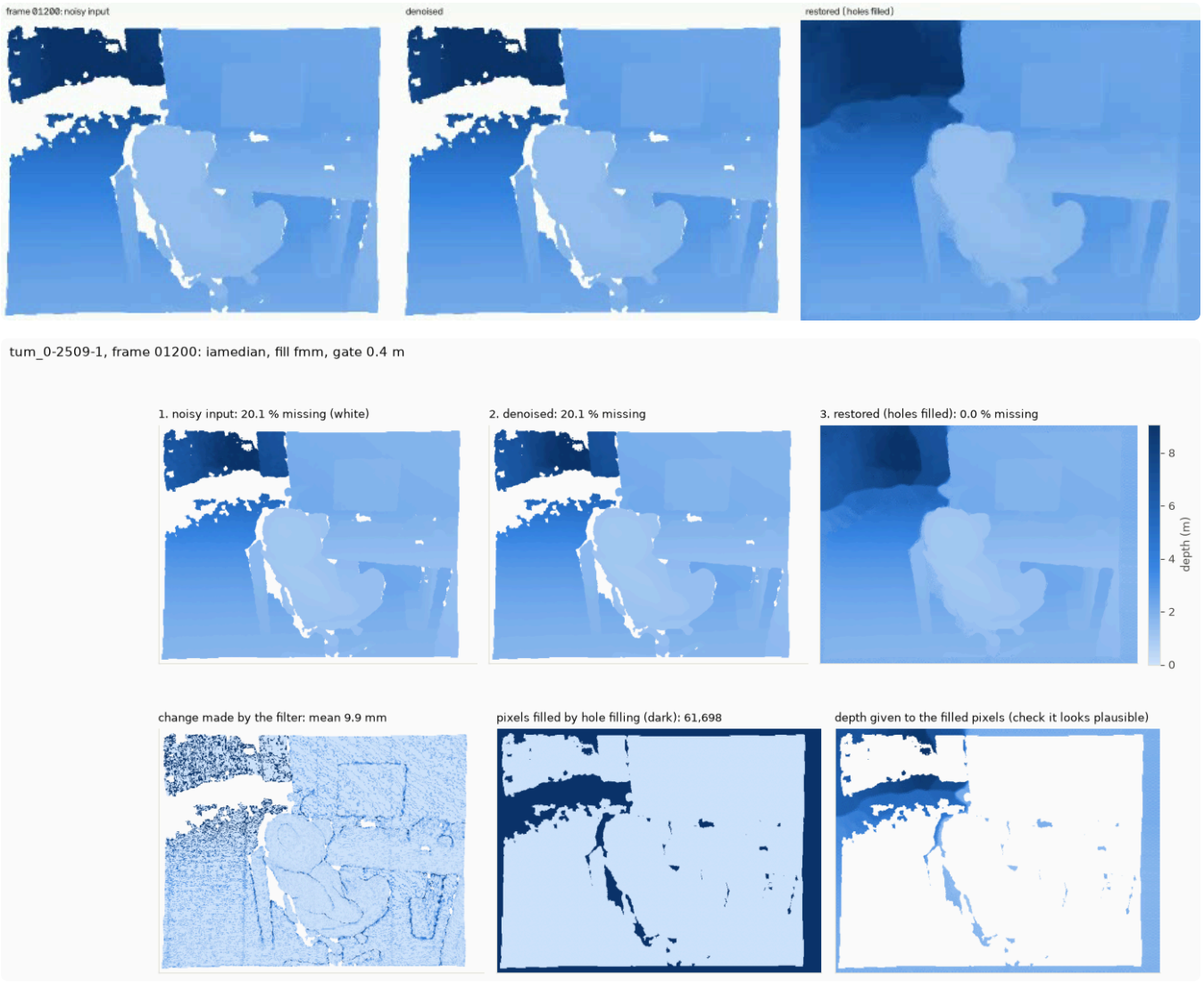
ICL, frame 775 in the preview video: noisy input, denoised, restored, and the error of the restored depth (light = small, dark = 5 cm or more).





Denoising removes most of the scattered error; what remains sits on object outlines. The filled pixels (mostly on those outlines) are visible as the slightly darker edges in the restored error map.

TUM, frame 1200 in the preview video: noisy input, denoised, restored.



On real data the missing areas are large (20 % of this frame): the band at the upper left and strips beside the chair. FMM extends the surrounding depth into them, so the restored image is complete, but those values are guesses that cannot be checked without true depth.

The videos (`outputs/process/icl_full/preview.mp4`, `tum_full/preview.mp4`) and the 3D viewers (`viewer_3d.html`) are in the project folder; they are regenerated by the commands above and not kept in git.

How to run it yourself

The code-only package `kinect_lite_pipeline.zip` (4.2 MB) runs on any PC with Python; sample frames are included, so no download is needed to try it.

Requirements: Python 3.10 or newer, about 1 GB of RAM per worker, no GPU. One command installs NumPy, OpenCV, SciPy, pandas, Matplotlib and Plotly:

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
```

Inputs it accepts

input	command
the sample frames	`python denoise/process.py --input samples/tum/depth`
one depth image	`python denoise/process.py --input frame.png`
a folder of depth images	`python denoise/process.py --input my_folder`
a depth video (8-bit, grey 255 = max depth)	`python denoise/process.py --input depth.avi --video-max-depth 5`
a dataset	`python denoise/process.py --dataset icl --frames 0:1509:1`
with true depth, to measure error	add `--truth folder_of_true_depth`

Depth PNGs are 16-bit, value / 5000 = metres, 0 = missing; `--png-scale 1000` reads millimetre PNGs. Video is lossy, so 16-bit PNGs are the better input.

What it writes (to `outputs/process/<input name>/`)

file	what it shows
`preview.mp4`	every frame side by side: noisy input, denoised, restored (and the error, when true depth is given)
`stages.png`	every stage of one frame in detail

file	what it shows
`summary.png`	missing pixels (and error) along the sequence
`viewer_3d.html`	one frame as rotatable 3D point clouds, opened in a browser
`frames.csv`	per-frame numbers
`restored/*.png`	the restored depth images, same format as the input

Changing the method: `--filter` (iamedian, median, gaussian_masked, gaussian_cv2, bilateral_masked, bilateral_cv2, none) and `--fill` (fmm, ns, region, none). The package was installed into a fresh environment and every command above was run to check it.

Limits and next steps

The pipeline is reliable at removing noise on ICL; filled pixels and real-sensor behaviour are the open points.

Limits

- **Filled pixels are guesses.** On ICL they average 132 mm off with FMM; Navier-Stokes was about four times more accurate in the earlier test (33 vs 130 mm). Use `--fill ns`, or `--fill none` when only measured depth should be trusted.
- **Simulated noise.** ICL's edge spikes likely come from how the noise was simulated, which favours the median.
- **No true depth on TUM.** On the real sensor we can show what the pipeline changes, not whether it is right.
- **One frame at a time.** Frames do not help each other; a moving camera's neighbouring frames could fill holes more reliably.
- **Speed.** About 0.35-0.4 s per frame in Python; live 30 fps would need a faster implementation.

Next steps

1. **ICP:** estimate the camera's motion from the restored depth, and compare raw, denoised and restored input against TUM's true camera path. This is the real-sensor test of the pipeline.
2. **Fill method:** switch the default to Navier-Stokes if ICP confirms its advantage.

3. **Learned methods (optional):** a trained depth-completion model could use scene knowledge the filters lack; this pipeline is the baseline it would have to beat.